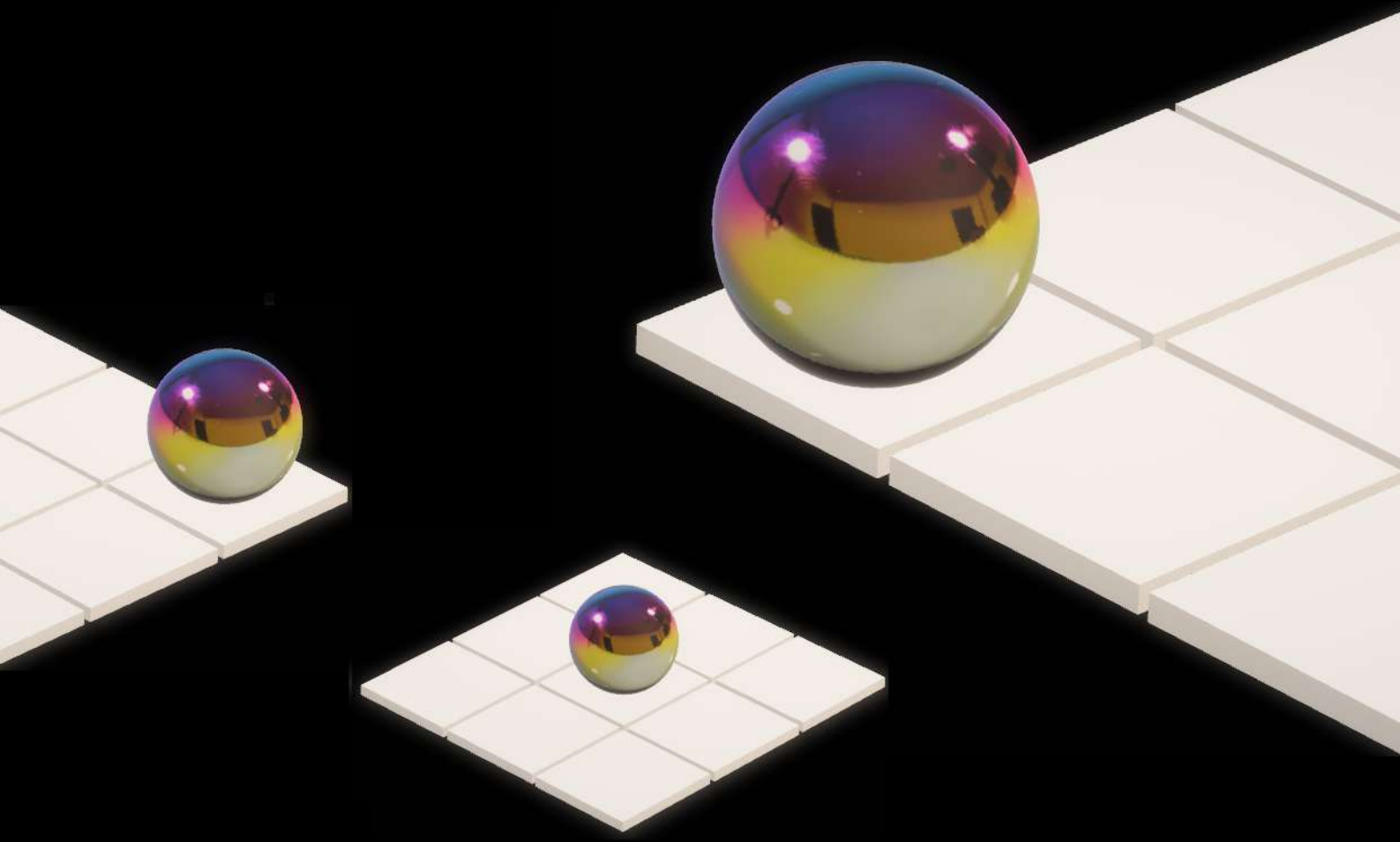
LOGIC COMPONENTS

LUKE NEO-SHELL



LUKE'S.CORP

NEO-Shell

完全コマンドリファレンスガイド

Logic-Software-NEO-Shell® バージョン I — コマンド紹介

Logic Components Corp · 著者 : Fogli Luca · 2024 – 2025

目次

1. はじめに・言語構造

2. 変数 — var / killvar / allvar

3. 式と計算 ONE FOR ALL! — "$"

4. 入力読み取り — in

5. 出力表示 — print / wait

6. ブロックと関数 — blk / vob

7. フロー制御 — if / elseif / else

8. ループ — for / while / loop / switch

9. 制御シグナル — break / continue / return

10. I/Oポート — out

11. EEPROM — eepset / eeplist / eeprun / eepcls / eepmem

12. I2Cプロトコル — iset / ifind

13. システム変数

14. 特殊コマンド — boot / Empty

15. 完全クイックリファレンス

A. 付録A — システムメッセージ（MSGbox）

1. はじめに・言語構造

NEO-Shellは、ATmega1284Pマイクロコントローラ向けに設計されたインタープリタ型スクリプト言語です。115200ボーのシリアルポートまたはI2Cバス経由でリアルタイムにコマンドを実行し、デバイスのEEPROMに永続的なプログラムを保存し、変数・ループ・条件分岐・ハードウェアI/Oを管理できます。

基本構文

セミコロン(;)は同一行上のコマンド間のセパレータであり、必須のターミネータではありません。単一のコマンドにはセミコロンは不要です。コードブロックは { } で囲みます。条件は ( ) で、ブロック値は [ ] で囲みます。

```
out d 13 1                          // 単一コマンド : ; 不要
out d 13 1 ; wait 500 ; out d 13 0  // 同一行の複数コマンド : ; で区切る
var A = 10; var B = 20; print $A    // 区切りの別例
if ($A > 5) { print "ok\n" }       // ブロック : ; はオプション
```

基本ルール

ルール	説明
セパレータ ;	同一行のコマンドを区切ります。単一コマンドには不要です。
変数の $	変数の値を読み取るには常に $名前 を使用します
ブロック { }	if、for、while、loop、switch の本体を区切ります
条件 ( )	if、while、for、switch の条件を囲みます
ブロック関数 [ ]	ブロック内のコマンド本体を囲みます:blk 名前 = [関数]
大文字小文字区別	変数名とコマンド名は大文字小文字を区別します
最大変数数	メモリ内に同時に256個の変数
最大I2C文字列	1回のI2Cメッセージあたり32バイト(ハードウェア制限)

2. 変数 — var / killvar / allvar

var — 変数の宣言と代入

新しい変数を作成するか、既存の変数の値を更新します。変数の型はシステムが自動的に検出します。

検出型	条件
int	値が整数の場合（例：10、-3、0）
float	値に小数部がある場合（例：3.14、-0.5）
char	値が1文字の場合（例：A、z、!）
string	値がその他の文字列の場合（例：Hello world!、on）

構文	説明
`var 名前 = 値`	直接値で変数を作成
`var A = 10, B = 2.05, C = !`	同一行での複数代入（カンマ区切り）
`var A = $B`	別の変数の値をコピー
`var $ポインタ = 値`	ポインタ変数に含まれる名前に代入（間接参照）

使用例

```
var Temperatura = 23  // int
var Tensione = 3.14   // float
var Lettera = A       // char
var Messaggio = CiaoMondo  // string
var X = 10, Y = 20, Z = 0  // 複数
var Doppio = $X       // X を Doppio にコピー
var Nome = pippo
var $Nome = 42        // 'pippo' = 42 を作成/更新（間接参照）
```

killvar — メモリからの変数削除

1つ以上の変数を削除します。引数なしの場合はすべてを削除します。カンマリストと @ によるレンジをサポートします。

構文	説明
`killvar`	全変数を削除（リセット）
`killvar 名前`	変数「名前」を削除
`killvar A,B,C`	A、B、C を削除
`killvar A@Z`	作成順でAからZまでの全変数を削除

```
killvar  // 全リセット
killvar Temp  // Temp を削除
killvar A,B,C  // 3つの変数を削除
```

allvar — メモリ内の変数表示

メモリ内に割り当てられた変数を表示します。注意：出力形式は呼び出し方によって異なります。

構文	出力	説明
`allvar`	名前 = 値	'名前 = 値' 形式で全変数を表示
`allvar 名前`	値	名前の値のみを表示（名前なし）
`allvar A,B,C`	値\n値\n値	A、B、Cの値のみを1行ずつ表示
`allvar A@Z`	値\n...	A-Zの範囲の値のみを表示（作成順）

⚠ パラメータ付き（名前、リスト、レンジ）のallvarは値のみを表示し、名前は表示しません。引数なしのallvarのみが「名前 = 値」を表示します。

```
allvar  // 全変数を名前 = 値で表示
allvar X  // 表示: 42 (Xの値のみ)
allvar A@C  // A、B、Cの値を1行ずつ表示
```

3. 式と計算 ONE FOR ALL! — "$"

$ を使って数学的/論理的な式を実行し、結果を変数に代入できます。演算子、数学関数、ネストされた括弧の式、$名前による変数読み取り、サフィックスによる結果型指定をサポートします。

```
$ NAME = expression;
```

算術演算子とビット演算子

演算子	説明	例	結果
+	加算	$A = 5 + 3	8
-	減算	$A = 10 - 4	6
*	乗算	$A = 3 * 4	12
/	除算	$A = 10 / 4 f	2.5
%	剰余（余り）	$A = 10 % 3	1
@	べき乗（右結合）	$A = 2@10	1024
&	ビットAND	$A = 1 & 0	0
|	ビットOR	$A = 1 | 0	1
^	ビットXOR	$A = 1 ^ 1	0
~	ビットNOT（単項前置）	$A = ~0	-1
!	論理NOT（単項前置）	$A = !0	1
++	インクリメント	$A++	A = A+1
--	デクリメント	$A--	A = A-1

特殊クランプ/内部比較演算子

注意：$ 式内の演算子 > < = は、従来の言語のように 0/1 を返しません。「数値クランプ」の動作をします：

演算子	実際の動作	例	結果
A > B	A > B なら B を返し、そうでなければ A を返す（上限クランプ）	$R = 100 > 50	50
A < B	A < B なら B を返し、そうでなければ A を返す（下限クランプ）	$R = 10 < 50	50
A = B	A == B なら B を返し、そうでなければ A を返す	$R = 5 = 5	5

⚠ 真偽値（true/false）の比較には if ($A > $B) 構造を使用してください。if/while の外で使われる > < = はクランプ演算子であり、比較演算子ではありません。

数学関数

関数	説明	例
$var = sin N	N度のサイン	$R = sin90 → 1
$var = cos N	N度のコサイン	$R = cos0 → 1
$var = tan N	N度のタンジェント	$R = tan45 → 1
$var = r2 N	Nの平方根	$R = r2144 → 12
$var = r3 N	Nの立方根	$R = r327 → 3
$var = log N,B	底BのNの対数	$R = log100,10 → 2
$var = map V,iMin,iMax,oMin,oMax	線形リマッピング	$R = map512,0,1023,0,100 → 50
$var = rnd MIN,MAX	乱数（TrueRandハードウェア）	$R = rnd1,100 → 1..99

i 三角関数（sin、cos、tan）はラジアンではなく度を受け取ります。

i rnd はTrueRandomシステムを使用し、疑似乱数システムではありません。

i map は線形リマッピングです：map(値, inMin, inMax, outMin, outMax)。

結果型指定子

最後のトークンとしてスペースで区切った文字（または数字の末尾に付加）を追加すると、結果の型を強制します：

サフィックス	型	例	出力
i（デフォルト）	int — 整数部	$A = 10 / 3 i	3
f	float — 小数	$A = 10 / 3 f	3.333333
c	char — ASCII文字	$A = 65c	'A'
s	string — 文字列	$A = 3.14 s	"3.14"

使用例

```
$A = 10 + 5     // A = 15 (int)
$B = $A * 2     // B = 30
$C = r2144      // C = 12（144の平方根）
$D = (10 + 20) / 3 f  // D = 10.0 (float)
$E = 2@8        // E = 256（2の8乗）
$F = sin90      // F = 1
$G = rnd1,100   // G = 乱数 1-99
$H = map512,0,1023,0,255  // H = ADC → PWM リマッピング
$I = log1000,10  // I = 3
$J = !0         // J = 1（論理NOT）
$K = ~255       // K = -256（ビットNOT）
$A++            // A をインクリメント
$A--            // A をデクリメント
```

i $ 式内でオペランドとして使用された単一のアルファベット文字はASCIIコードに自動変換されます。例：$X = A → X = 65。

4. 入力読み取り — in

in コマンドはハードウェアまたはユーザーから値を読み取り、変数に代入します。常に $ 式の中で使用します：

```
$VARNAME = in TYPE SOURCE PIN;
```

型	ソース	説明
$var = in D ピン	ピン / ハードウェア	デジタルピンを読み取り（0または1）。ピンをINPUTとして設定。
$var = in A ピン	ピン / ハードウェア	アナログピンを読み取り（0-1023、10ビット）。
$var = in K	デフォルト	シリアルまたはI2Cから1文字を読み取り（どちらか先に来た方）
$var = in KS	シリアル	シリアルポートのみから1文字を読み取り
$var = in KI	I2C	I2Cのみから1文字を読み取り
$var = in T	デフォルト	シリアルまたはI2Cから1行テキストを読み取り（\nまで）
$var = in TS	シリアル	シリアルポートのみから1行テキストを読み取り
$var = in TI	I2C	I2Cのみから1行テキストを読み取り

ℹ in K と in T はブロッキングです：入力が来るまで実行が停止します。in K のタイムアウトは5秒、in T は10秒です。

ℹ in K と in T の前に、残留文字の読み取りを避けるためシリアルバッファが自動的にフラッシュされます。

使用例

```
$Stato = in D 13   // デジタルピン13を読み取り
$Pot = in A 0      // アナログピンA0を読み取り（0-1023）
$Tasto = in K      // シリアルまたはI2Cからキー入力を待つ
$TastoSerial = in KS  // シリアルのみから待つ
```

```
$Riga = in T       // テキスト行を待つ（Enterまで）
$RigaI2C = in TI   // I2Cのみから行を待つ

// 例：インタラクティブメニュー
print "1でLED ON、2でLED OFFを押してください:\n"
$Scelta = in KS
if ($Scelta == 1) { out d 13 on; }
if ($Scelta == 2) { out d 13 off; }

// 例：リマッピング付きアナログ読み取り
$Valore = in A 0
$PWMval = map $Valore,0,1023,0,255
out a 9 $PWMval   // ハードウェアPWMに値を適用
```

5. 出力表示 — print / wait

print — シリアルまたはI2Cへのテキストと値の表示

シリアルまたはI2Cデバイスに出力します。引用符の有無に基づいて2つのモードを区別します：

コンテキスト	動作
引用符内 "X"	$名前は変数値に展開されます。\n \t \r シーケンスが有効です。
引用符外 X	$NEO経由で数学式として評価されます。変数、演算子、関数。
混在 "X" Y "Z"	引用符内外の部分が結合されて1つの出力になります。
I2C @@ "X"	print @アドレス@ 内容 — 2つの @ が必要。32バイト制限。

引用符内で有効なエスケープシーケンス

シーケンス	生成文字
\n	改行（LF、0x0A）

シーケンス	生成文字
\t	タブ（TAB、0x09）
\r	キャリッジリターン（CR、0x0D）
\X	その他の文字Xはそのまま表示（例:\" は引用符を表示）

⚠ I2C構文は:print @アドレス@ "テキスト" で、アドレスの前後に2つの @ が必要です。

使用例

```
print "ようこそ！\n"
var T = 23
print "温度: $T 度\n"
$S = 10 + 20
print "合計 = " $S "\n"  // 混在：テキスト + 式
print $T * 2  // 引用符外：46を表示
print @0x08@ "out d 13 on"  // I2C経由で送信
```

wait — 実行の一時停止（ミリ秒）

指定されたミリ秒数だけ実行を停止します。パラメータは直接値または$付き変数が使用できます。

```
wait 1000  // 1秒一時停止
wait 500   // 500ms一時停止
var T = 2000
wait $T    // 変数から2秒一時停止
```

6. ブロックと関数 — blk / vob

NEO-Shellでは関数に似た再利用可能なコードブロックを定義できます。ブロックはblkで定義され、変数に保存されます。内部で宣言されたvobプレースホルダーを通じてパラメータを受け取ることができます。

blk — コードブロック（関数）の定義

[ ] で囲まれたコードブロック全体を値として持つ変数を作成します。ブロックはその名前をコマンドとして書くだけで呼び出せます。vobが含まれている場合、呼び出し時にパラメータを受け取ります。

構文	説明
`blk 名前 = [ ... ]`	直接名前でブロックを作成
`blk $名前変数 = [ ... ]`	名前変数の値を名前として使用してブロックを作成（間接参照）

```
blk Saluta = [ print "ブロックからこんにちは！\n" ]
Saluta  // ブロックを呼び出す
```

vob — ブロックの仮パラメータ（Variable Of Block）

パラメータを受け取ることを示すためにblk内で宣言されます。オプションのセパレータ（名前の後の二重引用符内）は次のパラメータを区切る文字/文字列を指定します。デフォルト：スペース。

構文	説明
`vob 名前`	スペースで区切られたパラメータ（デフォルト）
`vob 名前 ","`	カンマで区切られたパラメータ
`vob 名前 "7"`	数字で区切られたパラメータ
`vob 名前 "%"`	パーセントで区切られたパラメータ

⚠「;」を使用すると、インタープリタが次の値を次のvobに関連付ける値としてではなく、実行するコマンドとして解釈するリスクがあります。

ℹ ブロックの最後のvobは、セパレータに関わらず常に残りの文字をすべて収集します。

例:2つのパラメータを持つブロック(スペース区切り)

```
blk Somma = [
  vob A;
  vob B;
  $R = $A + $B;
  print "結果: " $R "\n"
]
Somma 10 20  // A=10, B=20 → 結果: 30
```

例:カスタム区切り

```
blk Info = [
  vob Nome ","  // 最初のパラメータ（カンマまで）
  vob Eta       // 2番目のパラメータ：残りの文字列
  print "名前: " $Nome " 年齢: " $Eta "\n"
]
Info Mario,25  // 名前=Mario、年齢=25
```

7. フロー制御 — if / elseif / else

NEO-Shellは完全な条件構造をサポートします。条件には比較演算子と論理演算子を使用します。比較は数値(浮動小数点許容値での数値比較)と文字列(辞書的比較)の両方で機能します。

演算子	意味	備考
==	等しい	数値（許容値0.0001）または辞書的
!=	等しくない	数値または辞書的
>	より大きい	
<	より小さい	
>=	以上	
<=	以下	
&&	論理AND	複数の条件を組み合わせる
||	論理OR	複数の条件を組み合わせる

if — 条件実行

```
if (condition) { instructions }
```

```
var X = 10
if ($X > 5) { print "X > 5\n" }
if ($X >= 10 && $X < 20) { print "10から19の間" }
if ($X == 10 || $X == 20) { print "10または20" }
```

elseif — 代替条件分岐

if（または別のelseif）の直後に続く必要があります。前のすべての分岐が偽の場合にのみ実行されます。

```
elseif (condition) { instructions }
```

else — デフォルト分岐（それ以外）

```
else { instructions }
```

完全な例 — if/elseif/else チェーン

```
var Voto = 75
if ( $Voto >= 90 ) { print "優秀！" }
elseif ( $Voto >= 70 ) { print "良い" }
elseif ( $Voto >= 60 ) { print "合格" }
else { print "不合格" }
```

8. ループ — for / while / loop / switch

for — 初期化、条件、インクリメント付きループ

条件内に ; で区切られた3つの部分を持つ古典的なループ。変数はvarで自動的に宣言されます。break、continue、returnをサポートします。

```
For ( i = 0; $i < 10; $i++) { instructions }
```

⚠ for は単純な計算式を使用します：論理演算子 && と || は for の条件では機能しません。複雑な条件には break を使った内部 if を使用してください。

```
// 0から9までカウント
for ( i = 0; $i < 10; $i++) { print $i "\n" }

// デクリメント付きループ
for ( k = 10; $k > 0; $k--) {
  out d 13 on
  wait 100
  out d 13 off
  wait 100
}
```

while — 入場条件付きループ

条件が真の間ブロックを繰り返します。条件は各反復の前に評価されます。

```
while (condition) { instructions }

var N = 0
while ( $N < 5 ) { print $N "\n" ; $N++ }
```

loop — 無限ループ

条件なしで無限にブロックを実行します。終了するには break または return を使用します。継続的な監視プログラムに最適です。

```
loop { instructions }

var C = 0
loop { $C++ ; print "Tick " $C "\n" ; wait 100 ; if ($C >= 60) { break
} }
// 60回後に終了
```

switch — 複数値の選択

変数の値を一連のcaseと比較します。一致するcaseがない場合、DEFAULTブロック（存在する場合）を実行します。各caseは通常、フォールスルーを避けるためbreakで終わります。

```
switch ($VARIABLE) {
  case VALUE1 { instructions; break }
  case VALUE2 { instructions; break }
  DEFAULT { instructions }
}
```

完全な例

```
var Giorno = 3
switch ($Giorno) {
  case 1 { print "月曜日" ; break }
  case 2 { print "火曜日" ; break }
  case 3 { print "水曜日" ; break }
  case 4 { print "木曜日" ; break }
  DEFAULT { print "週末" }
}
```

9. 制御シグナル — break / continue / return

break — 現在のループまたはswitchを終了

存在するループ（for、while、loop）またはswitchの実行を即座に中断します。実行はブロックの次の命令から再開されます。

```
var I = 0
while (1 == 1) { $I++ ; if ($I >= 5){ break } }
print "終了 I = " $I "\n"  // 表示: 5
```

continue — 次のループ反復にスキップ

現在の反復を中断して次に移ります。forでは、条件に戻る前にインクリメントを実行します。

```
for ( k = 0; $k < 10; $k++) { if ($k == 5) { continue }; print $k " "
}
// k=5のときスキップ
// Output: 0 1 2 3 4 6 7 8 9
```

return — 現在のblkブロックを終了（オプション値あり）

現在のblkブロックの実行を終了し、呼び出し元に制御を戻します。オプションで $return システム変数に保存される値を返すことができます。

構文	説明
`return;`	値なしでblkを終了
`return 値;`	blkを終了し「値」を$returnに保存
`return $EXPR;`	blkを終了し$EXPRの値を$returnに保存

⚠ return は blk にスコープされます：shouldReturnフラグはブロック実行終了後に自動的にリセットされます。blk内のreturnは外部に伝播しません — 呼び出しコードは通常通り続行します。

```
blk Massimo = [ vob A ; vob B ; if ($A > $B) { return $A }; return $B
]
Massimo 10 20
print "Max = " $return "\n"  // 表示: Max = 20
```

10. I/Oポート — out

out コマンドはすべてのハードウェア出力を管理します。出力タイプはoutの直後の文字（d、a、t、s、p）です。パラメータ値は定数または$変数が使用できます。

コマンド	説明	例
`out d ピン on`	デジタル出力 HIGH	out d 13 on
`out d ピン off`	デジタル出力 LOW	out d 13 off
`out d ピン hi`	デジタル出力 HIGH（別名）	out d 13 hi
`out d ピン lo`	デジタル出力 LOW（別名）	out d 13 lo
`out d ピン 1`	デジタル出力 HIGH（数値）	out d 13 1
`out a ピン 値`	ハードウェアPWM analogWrite（0-255）	out a 9 128
`out t ピン 周波数`	トーン：ピンで継続音	out t 8 440
`out t ピン 周波数 長さ`	ミリ秒単位の長さ付きトーン	out t 8 440 500
`out t ピン 0`	トーンを無効化（noTone）	out t 8 0
`out s ピン 角度`	サーボ：N度に位置決め（0-180）	out s 6 90
`out p ピン 周波数 デューティ`	ソフトウェアPWM：Hz、%	out p 5 1000 50
`out p ピン 周波数 デューティ 長さ`	ミリ秒単位の長さ付きソフトウェアPWM	out p 5 1000 50 3000
`out p ピン 0 0`	ピンのソフトウェアPWMを停止	out p 5 0 0

ℹ out p はソフトウェアPWMを生成します：任意のピンで最大5つの同時信号をサポートします。周波数とデューティサイクルは完全に自由です。freq=0またはduty=0で信号が停止します。

パラメータでの変数使用

```
var PIN = 13 , Stato = 110
out d $PIN on    // 変数からPINを使用
out a 9 $Stato   // PWM値としてStatoを使用
```

実用例

```
// LEDの点滅
loop { out d 13 on ; wait 500 ; out d 13 off ; wait 500 }

// サーボスイープ
for ( pos = 0 ; $pos <= 180 ; $pos++ ) { out s 13 $pos; wait 15 }
```

```
// ソフトウェアPWM 2kHz、30%デューティ、5秒
out p 5 2000 30 5000;
```

11. EEPROM — eepset / eeplist / eeprun / eepcls / eepmem

EEPROMにより、電源を切っても残るNEO-Shellプログラムを保存できます。行は1から番号付けられます。1行目はbootマーカー用に予約されています。プログラムはeeprunで実行されます。

eepset — EEPROMライン書き込み・編集・操作

構文	説明
`eepset`	インタラクティブエディタモード — 行を入力、「end」で終了
`eepset N テキスト`	行Nにテキストを Nに書き込み/置換
`eepset +N テキスト`	位置Nに新しい行としてテキストを挿入（後続行をシフト）
`eepset +N`	位置Nに空行を挿入
`eepset => A B`	行Aと行Bの内容を入れ替え
`eepset => A@B C@D`	行グループA-Bとグループ C-Dを入れ替え
`eepset <= A B`	=>と同じだが逆方向（B → A）
`$ 変数 = eepset N`	EEPROMから行Nを読み取り変数に保存

ℹ eepsetは書き込むテキスト内の$名前変数を自動的に置換します。展開なしでリテラルの$を書くには$$名前を使用します — 二重$$は単一$に縮小されます。

ℹ 読み取り $ 変数 = eepset N は $ コマンド構文を使用します（varではない）。例：$ Line = eepset 3;

```
eepset  // インタラクティブエディタを起動
```

インタラクティブエディタは挿入された各行のリアルタイムフィードバックを表示します：

```
TEXT << Used: 120 Free: 903 bytes >>
```

エディタを終了するには「end」と入力します。エディタはシリアルポートとI2Cの両方から入力を受け付けます。

この方法で、関係する行の番号を最初に入力し、次に新しいテキストまたはコマンドを入力することで、コードの個々の文字列を変更することもできます。

```
eepset 1 boot           // 1行目 = bootマーカー
eepset 2 var LED = 13   // 2行目にコードを書き込み
eepset +3 print "inserted"  // 位置3に新しい行を挿入
eepset +5               // 位置5に空行を挿入
eepset => 2 5           // 2行目と5行目を入れ替え
eepset <= 2 5           // 逆方向に入れ替え
eepset 2 print $$Var    // リテラルで書き込み: print $Var
$ Riga = eepset 2       // 変数Rigaに2行目を読み取り
```

eeplist — EEPROMコンテンツの表示

構文	説明
`eeplist`	番号と内容付きで全行を表示
`eeplist N`	N行目のみを表示
`eeplist N@M`	NからMまでの行を表示（両端含む）
`eeplist N,M,K`	N、M、Kの行を表示（任意リスト）

```
eeplist         // 全表示
eeplist 1@5     // 1-5行目を表示
eeplist 2,4,6   // 2、4、6行目を表示
```

eeprun — EEPROMプログラムの実行

構文	説明
`eeprun`	全行を実行（1行目'boot'はスキップ）
`eeprun N`	N行目のみを実行
`eeprun N>`	N行目から最後まで実行

構文	説明
`eeprun N@M`	NからMまでの行を実行（両端含む）
`eeprun N,M,K`	N、M、Kの行をその順に実行

```
eeprun        // プログラムを実行
eeprun 5      // 5行目のみ実行
eeprun 2@8    // 2-8行目を実行
eeprun 3>     // 3行目から実行
eeprun 2,5,7  // 2、5、7行目を実行
```

eepcls — EEPROMラインのクリア

EEPROMから行をクリアします。2つのクリアモードがあります：

モード	構文	動作
クリア（デフォルト）	eepcls N	行を'Empty'に置換 — 行は残るが空
物理削除	eepcls N!	行を完全削除 — 後続行がコンパクト化
レンジクリア	eepcls N@M	NからMの行をクリア
レンジ削除	eepcls N!@M!	NからMの行を物理削除
混在リスト	eepcls N,M!,K	Nクリア、M削除、Kクリア
完全フォーマット	eepcls	EEPROM全体をクリア（バイト単位）

⚠ eepcls は !なしで行をクリア（'Empty'を書き込み）します。!付きで物理的に削除し、後続行をコンパクト化します。物理削除後は行番号が変わります。

```
eepcls        // EEPROM全フォーマット
eepcls 5      // 5行目 → 'Empty'
eepcls 5!     // 5行目を物理削除
eepcls 2@4    // 2,3,4行目 → 'Empty'
eepcls 2!@4!  // 2,3,4行目を削除
```

eepmem — EEPROMメモリ統計

EEPROMの使用バイト数と残り空きバイト数を表示します。

```
eepmem  // 例: MEM Bytes used 120 / MEM Bytes free 903
```

12. I2Cプロトコル — iset / ifind

NEO-ShellはI2C MASTERとSLAVEとして同時に動作します。スレーブとしてデフォルトアドレス0x12に応答します。マスターとして他のデバイスにコマンドを送信できます。1メッセージの制限は32バイトです。

iset — 送信用I2Cターゲットアドレスの設定

print @...@が使用するI2C送信先アドレスを変更します。16進数（0xNN）または10進数形式のアドレスを受け付けます。新しいスレーブアドレスでWireバスを再初期化します。

構文	説明
`iset 0x08`	ターゲットを0x08に設定（NEOデフォルト）
`iset 0x12`	ターゲットを0x12に設定
`iset $変数`	変数の値をアドレスとして使用

```
iset 0x08   // ターゲットを0x08に変更
```

i 有効アドレス：0x08 – 0x77。この範囲外では「Addr Invalid」エラーが発生します。

ifind — I2Cバス スキャン

パラメータなしで完全スキャン（0x08–0x77）を実行し、見つかったすべてのデバイスを表示します。アドレスパラメータ付きの場合はそのデバイスのみを確認します。

構文	説明
`ifind`	完全スキャン — 見つかったすべてのアドレスを表示
`ifind 0x08`	アドレス0x08が存在するか確認
`ifind $変数`	$変数に含まれるアドレスを確認

I2Cアドレスが検索されると、NEO-Shellは「$I2C」変数を設定します。アドレスが存在して接続されている場合、値は1（TRUE）になり、そうでない場合は0（FALSE）になります。

```
ifind       // 完全スキャン
ifind 0x08  // 0x08を確認
if ($I2C == 1) { print "デバイスが見つかりました！" }
```

13. システム変数

NEO-Shellは実行中にシステムによって自動的に更新される特殊変数を管理します。

変数	型	説明
$return	任意	最後に「return 値」を使ったblkが返した値
$I2C	int (0/1)	ifindで設定：デバイスが見つかれば1、そうでなければ0
$NEOin	string	前のコマンドのテキスト（新しいコマンド実行前に更新）
$NEO	計算値	引用符外の式を評価するためにprintが使用する内部一時変数

ℹ $NEOinは現在のコマンド実行前に更新されます：常に直前のコマンドを含み、今起動したコマンドではありません。

使用例

```
// $returnの使用
blk Rad = [ vob X; return r2$X ]
Rad 144
print "平方根 = " $return "\n"  // 表示: 12


// $I2Cの使用
ifind 0x08
if ($I2C == 1) {
  print @0x08@ "NEO-Shellターミナルに接続されています！"
}


// $NEOinの使用
print "最後のコマンド: " $NEOin "\n"
```

14. 特殊コマンド — boot / Empty

boot — EEPROMの自動起動マーカー（1行目）

実行可能なコマンドではなく、EEPROMの1行目に配置する必要がある特殊マーカーです。電源投入時、NEO-Shellは1行目を確認し、bootが含まれていれば後続の行を自動的に実行します。

1行目の構文	起動時の動作
`boot`	2行目以降を全実行（eeprunと同等）
`boot N`	N行目以降を実行
`boot N@M`	NからMの行を実行
`boot N,M,K`	N、M、Kの行を実行

ℹ 1行目の「boot」比較は大文字小文字を区別しません：BOOT、Boot、boot はすべて機能します。

```
// 起動時のLED点滅プログラム：
eepset 1 boot
eepset 2 var LED = 13
eepset 3 loop { out d $LED on ; wait 500 ; out d $LED off ; wait 500 }
```

Empty — EEPROMの空行マーカー

実行可能なコマンドではありません。!修飾子なしでeepcls によって行が消去された際（論理削除）にインタープリタが自動的に書き込みます。インタープリタはeeprun中にEmptyとマークされたすべての行を自動的にスキップします。

```
eepcls 2    // 2行目が'Empty'になる
eeplist 2   // 表示: 2 > Empty
eeprun      // 2行目は自動的にスキップ
```

付録A — システムメッセージ（MSGbox）

NEO-Shellは、状態・エラー・確認を通知するために番号付きメッセージの内部ライブラリを使用します。これらは使用中にシリアルモニターに自動的に表示されます。完全な表：

番号	シリアルメッセージ	表示タイミング
0	Logic Components(C) / NEO-Shell Version I / Copyright...	デバイス起動時（バナー）
1	>>	インタラクティブプロンプト（次のコマンド入力待ち）

番号	シリアルメッセージ	表示タイミング
2	Run	eeprun実行開始
3	Stop	実行終了/停止
4	Error	一般的なフォーマットまたは構文エラー
5	Ok	操作成功（一般）
6	Wait	システム待機中
7	!	一般的な警告
8	?	入力要求または不確実
9	Cleared	変数メモリまたはEEPROMクリア
10	Saved	データ正常保存（例：eepset後）
11	Format Type?	blk内で認識できないフォーマット
12	Typo	入力ミス/認識できないコマンド
13	Mem Empty!	変数メモリが空
14	Mem Tall!	変数メモリが満杯（256変数に達した）
15	###	EEPROMフォーマット進捗バー
16	EEProm Empty!	EEPROMが空（行なし）
17	EEProm Tall!	EEPROMが満杯
18	MEM Bytes used	使用バイト数のプレフィックス（eepmem）
19	MEM Bytes free	空きバイト数のプレフィックス（eepmem）
20	Not Exist!	変数またはEEPROM行が見つからない
21	End!	bootまたはeeprunの実行終了
22	ok	I2C送信成功（i2cSendData）
23	Error: NACK Addr	I2C：受信者がアドレスを確認しなかった
24	Error: NACK Data	I2C：受信者がデータを受け入れなかった
25	Error: Timeout	I2C：送信タイムアウト

番号	シリアルメッセージ	表示タイミング
26	Error: Data Long	I2C：データが長すぎる（>32バイト）
27	Addr: 0x	I2Cアドレス表示プレフィックス
28	No devices	ifind：I2Cデバイスが見つからない
29	Scanning...	ifind：バス スキャン進行中
30	0x	ifind中に見つかったアドレスのプレフィックス
31	Addr Invalid	I2Cアドレスが範囲外（有効：0x08-0x77）

15. 完全クイックリファレンス

コマンド / 構文	使用方法	カテゴリ
`var X = 値`	変数を宣言/更新	変数
`var A=1, B=2, C=3`	複数宣言	変数
`killvar [X / A,B / A@Z]`	変数を削除	変数
`allvar [X / A,B / A@Z]`	変数を表示	変数
`$ X = 式`	数学計算、結果をXに	式
`$ X++ / $ X--`	インクリメント / デクリメント	式
`$ X = A > B (clamp)`	クランプ：A>BならBを返す	式
`$ X = A < B (clamp)`	クランプ：A<BならBを返す	式
`$ X = rndMIN,MAX`	TrueRandomハードウェア乱数	式
`$ X = r2N / r3N`	平方根 / 立方根	式
`$ X = sinN / cosN / tanN`	度数での三角関数	式
`$ X = logN,B`	底Bの対数	式
`$ X = mapV,i0,i1,o0,o1`	線形リマッピング	式
`$ X = A@B`	べき乗：AのB乗	式

`$ X = !V / ~V`	論理NOT / ビットNOT	式
`$ X = inDPIN`	デジタルピン読み取り(0/1)	入力
`$ X = inAPIN`	アナログピン読み取り 0-1023	入力
`$ X = inK[S/I]`	1文字読み取り(S=シリアル、I=I2C)	入力
`$ X = inT[S/I]`	1行テキスト読み取り(S=シリアル、I=I2C)	入力
`print "テキスト"`	シリアルに出力	出力
`print @0xNN@ "テキスト"`	I2C経由で送信(2つの@が必要)	出力
`wait ミリ秒`	ミリ秒単位で一時停止	出力
`blk N = [ ... ]`	ブロック/関数Nを定義	ブロック
`vob 名前 ["sep"]`	blkの仮パラメータ	ブロック
`N パラメータ`	パラメータ付きでブロックNを呼び出す	ブロック
`if (条件) {...}`	条件実行	フロー
`elseif (条件) {...}`	代替条件	フロー
`else {...}`	elseブランチ	フロー
`for (初期化;条件;増分) {}`	古典的forループ	ループ
`while (条件) {}`	whileループ	ループ
`loop {}`	無限ループ	ループ
`switch ($X) { case ... }`	複数選択	ループ
`break`	ループ/switchを終了	制御
`continue`	次のループ反復へ	制御
`return [値]`	blkを終了、オプション値を$returnに	制御
`out d PIN on/off/hi/lo`	デジタル出力	I/O
`out a PIN 値`	ハードウェアPWM analogWrite 0-255	I/O
`out t PIN 周波数 [長さ]`	トーン(ピンで音)	I/O
`out s PIN 角度`	サーボモーター 0-180	I/O

out p PIN f d [長さ]	ソフトウェアマルチチャンネルPWM（最大5）	I/O
eepset N テキスト	EEPROMライン書き込み	EEPROM
eepset +N [テキスト]	ラインを挿入（テキストはオプション）	EEPROM
eepset => A B	AとBのラインを入れ替え	EEPROM
eepset <= A B	逆方向に入れ替え	EEPROM
$ X = eepset N	ラインNを変数Xに読み取り	EEPROM
$$名前 in eepset	$名前をリテラルで書き込み	EEPROM
eeplist [N / N@M / N,M]	EEPROMラインを表示	EEPROM
eeprun [N / N> / N@M]	EEPROMラインを実行	EEPROM
eepcls [N / N! / N@M]	クリア（デフォルト）または物理削除（!付き）	EEPROM
eepmem	EEPROMバイト統計	EEPROM
iset 0xNN	I2Cターゲットアドレス設定（0x08-0x77）	I2C
ifind [0xNN]	バス スキャン/確認 → $I2C	I2C
$return	blkの最後のreturnの値	システム
$I2C	1=見つかった / 0=見つからない（ifindで更新）	システム
$NEOin	前のコマンドのテキスト	システム
$NEO	内部一時変数（printで使用）	システム
boot [N / N@M / N,M,K]	EEPROMの自動起動（1行目）	特殊
Empty	論理削除されたラインのマーカー	特殊